



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Medfinder: Web de directorio médico

INF331 - Pruebas de Software

Fecha:	30 de junio de 2023
Autores:	Cristian Bruna, 201630019-8 Roberto Covarrubias, 201804181-5
Profesor:	Oscar Reyes
Paralelo:	1
Link a GitHub:	Medfinder-jenkins
Link a video:	Presentación Medfinder

Índice

Índice de figuras	1
1 Introducción	1
2 Descripción del trabajo	2
3 Scripts de prueba	3
3.1 Estrategia de pruebas utilizadas	3
3.2 Definición de Pruebas	3
3.2.1 Pruebas unitarias/integración	3
3.2.2 Pruebas de Valores límite/Supuestos	3
3.3 Procedimiento de ejecución de pruebas	3
3.3.1 Pruebas unitarias del sistema CRUD	3
3.3.2 Pruebas unitarias de Valores Límite/ Supuestos	4
4 Problemas encontrados y soluciones	6
5 Conclusión	7

1. Introducción

El centro médico "Salud y Bienestar" ofrece diversos servicios de salud con un equipo de profesionales médicos y enfermeras en diferentes áreas como medicina general y pediatría.

Muchos pacientes del centro médico tienen dificultades para encontrar especialistas médicos adecuados y desconocen la ubicación de los especialistas cercanos. Además, algunos pacientes desean conocer los antecedentes y credenciales de los médicos antes de hacer una cita.

Para resolver este problema, el centro médico en este proyecto desarrollará una web de directorio médico que incluirá una lista de todos los especialistas disponibles en la clínica, sus áreas de especialización y permitirá a los pacientes buscar especialistas por especialidad, nombre y ubicación geográfica, además de dar acceso a la visualización de las credenciales del médico.

Dada la relevancia de la estética se utilizará React como tecnología principal de desarrollo Front-End, Django Rest Framework para montar el Back-End manejando SQLite como la base de datos del proyecto. En secciones posteriores, se definirán los supuestos, consideraciones y pruebas realizadas para asegurar la calidad del software desarrollado.

Además del desarrollo de la web de directorio médico, tenemos la misión de armar un sistema CI/CD (Integración Continua/Entrega Continua). Este enfoque consiste en preparar casos de prueba exhaustivos, configurar un entorno de Integración Continua (CI), generar un release de la aplicación utilizando Jenkins y ejecutar un conjunto de pruebas de interfaz automatizadas utilizando Selenium y Jenkins. Con la implementación de este sistema, buscamos asegurar la calidad del software desarrollado y garantizar una entrega eficiente y confiable de las nuevas versiones de la aplicación.

2. Descripción del trabajo

En primer se agregó al stack de tecnologías a utilizar con respecto a la presentación 1, en este trabajo se añadió Jenkins ejecutar las pruebas de forma automática conectándose con el repositorio de GitHub que contenía el proyecto.

La herramienta de testing ahora paso a ser una interfaz de Selenium, necesitando así modificar las pruebas para adaptarse al nuevo formato que deben seguir creando así una pipeline en Jenkins y montando los servicios locales en la red utilizando Ngrok para la ejecución automática de pruebas.

Se mantuvo la configuración GitFlow dentro de la herramienta GitHub en este trabajo. Se mantuvo una gestión de proyecto dentro de Jira.

Finalmente se desarrolló el entorno de ejecución automática y la creación de pruebas dentro de Jenkins. Debido a la falta de tiempo no se logró solucionar un error dentro de Jenkis con una incompatibilidad de uso en el entorno local, lo cual género que no se pudiera demostrar en vídeo la ejecución, pero tanto las pruebas, como el entorno y el levantamiento del proyecto fue exitoso.

Se agrego en conjunto con el PDF de este informe una copia del pipeline utilizado, este es llamado pipeline.xml. Enlaces de herramientas utilizadas dentro del proyecto:

- Link Slack integrado con GitHub, Jira y Jenkins
- Link Github
- Link Jira

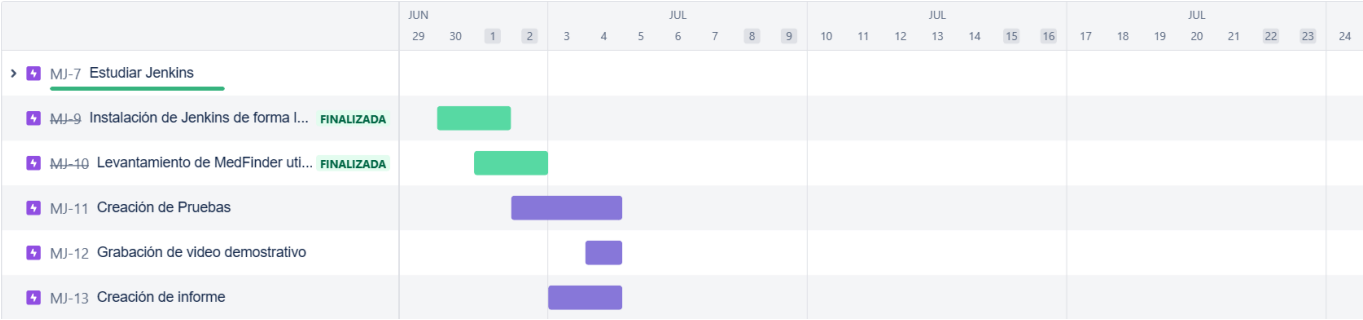


Fig. 1: Carta Gantt utilizada para la gestión de este trabajo.

3. Scripts de prueba

Las pruebas realizadas en este trabajo son de la misma naturaleza que las realizadas en la presentación 1, pero estas montadas y realizadas con framework Selenium.

3.1. Estrategia de pruebas utilizadas

Para definir las pruebas identificamos las siguientes estrategias:

1. Valores límite:
 - 1.1 Cantidad de caracteres en los campos.
 - 1.2 Edad aceptada (1-99) años.
2. Supuestos:
 - 2.1 Rut válido.
 - 2.2 Campo respeta formato pedido.
3. Pruebas funcionales.
4. Pruebas de Integración.

3.2. Definición de Pruebas

3.2.1. Pruebas unitarias/integración

- Abrir información de un médico.
- Realizar Login (Valid login admin).
- Crear médico (forma válida).
- Crear médico (forma no válida).
- Eliminar un médico.
- Editar Médico (forma válida).
- Editar Médico (forma no válida).
- Buscar médico por especialización.

3.2.2. Pruebas de Valores límite/Supuestos

- Rut válido/no válido (Validado por fórmula).
- Rut válido/no valido (Existe/No existe).
- Edad válida/no válida (0,1,100).
- Nombre válido/no válido (Largo).
- Nombre válido/no válido (Con espacios).
- Apellido válido/no válido (Largo).
- Apellido válido/no válido (Con espacios).
- Especialidad válida/no válida (espacios/largo).
- Credencial válida/no válida.
- Ubicación válida/no válida.

3.3. Procedimiento de ejecución de pruebas

3.3.1. Pruebas unitarias del sistema CRUD

P1: Prueba unitaria de creación de registro:

- 1.1 Se crea un registro ficticio y se llama al método de creación correspondiente.
- 1.2 Se verifica que el registro se haya guardado correctamente en la base de datos.
- 1.3 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P2: Prueba unitaria de lectura de registro:

- 2.1 Se agrega un registro ficticio a la base de datos.
- 2.2 Se llama al método de lectura correspondiente y se proporciona el ID del registro creado anteriormente.
- 2.3 Se verifica que el registro devuelto tenga los mismos valores que el registro creado anteriormente.

P3: Prueba unitaria de actualización de registro:

- 3.1 Se crea un registro ficticio y se guarda en la base de datos.
- 3.2 Se modifica algún campo del registro creado anteriormente y se llama al método de actualización correspondiente.
- 3.3 Se verifica que los cambios se hayan guardado correctamente en la base de datos.
- 3.4 Se verifica que los datos del registro actualizado sean los mismos que los proporcionados en la entrada.

P4: Prueba unitaria de eliminación de registro:

- 4.1 Se crea un registro ficticio y se guarda en la base de datos.
- 4.2 Se llama al método de eliminación correspondiente y se proporciona el ID del registro creado anteriormente.
- 4.3 Se verifica que el registro se haya eliminado correctamente de la base de datos.
- 4.4 Se verifica que el registro eliminado ya no esté presente en la base de datos al intentar leerlo.

3.3.2. Pruebas unitarias de Valores Límite/ Supuestos

P5: Validar Rut (Según fórmula)

- 5.1 Se ingresa un registro ficticio de un médico con un rut inválido (Según fórmula).
- 5.2 Se verifica que no se haya realizado el post del registro al Back-End.
- 5.3 Se ingresa un registro ficticio de un médico con un rut válido (Según fórmula).
- 5.4 Se verifica que se haya realizado el post del registro al Back-End.
- 5.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P6: Validar Rut (Existe o no Dentro de la Base de Datos)

- 6.1 Se ingresa un registro ficticio de un médico con un rut que ya exista dentro de la Base de Datos.
- 6.2 Se verifica que el sistema entregue feedback de que el rut proporcionado está actualmente en uso.
- 6.3 Se ingresa un registro ficticio de un médico con un rut válido (Según fórmula), y que no exista en la Base de Datos.
- 6.4 Se verifica que se haya realizado el post del registro al Back-End.
- 6.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P7: Validar Edad

- 7.1 Se ingresa un registro ficticio de un médico con una edad igual a 0.
- 7.2 Se verifica que el sistema entregue feedback de que la edad entregada no es correcta.
- 7.3 Se ingresa un registro ficticio de un médico con una edad igual a 100.
- 7.4 Se verifica que el sistema entregue feedback de que la edad entregada no es correcta.
- 7.5 Se ingresa un registro ficticio de un médico con una edad igual a 1.
- 7.6 Se verifica que se haya realizado el post del registro al Back-End.
- 7.7 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P8: Validar nombre/apellido/especialidad

- 8.1 Se ingresa un registro ficticio de un médico con un nombre con largo mayor a 50 caracteres.
- 8.2 Se verifica que el sistema no permita ingresar más de 50 caracteres al campo.
- 8.3 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres con un espacio al inicio.
- 8.4 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.
- 8.5 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que contenga números.
- 8.6 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.
- 8.7 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que contenga un tilde.

8.8 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.

8.9 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que no contenga tildes ni espacios al inicio.

8.10 Se verifica que se haya realizado el post del registro al Back-End.

8.11 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P9: Validar Ubicacion/Credencial

9.1 Se ingresa un registro ficticio de un médico con ubicacion inválida (No formato url)

9.2 Se verifica que el sistema entregue feedback de que la ubicacion entregado no es correcto.

9.3 Se ingresa un registro ficticio de un médico con ubicacion válida (Formato url).

9.4 Se verifica que se haya realizado el post del registro al Back-End.

9.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

4. Problemas encontrados y soluciones

Dentro de los problemas encontrados, fue el desconocimiento de las herramientas. Desconocíamos Jenkins y su funcionamiento e integración con el resto, pero con un par de videos tutoriales lo pudimos sobrellevar.

Luego vino el desconocimiento de Selenium, ya que creíamos que era en Java, pero luego supimos que se podía utilizar cualquier lenguaje de programación así que lo cambiamos a python por simplicidad y conocimiento.

También hubo desconocimiento de Ngrok, al ser el tier free no podíamos exportar más de 1 puerto a la vez, pero con un archivo de configuración udimos sacar los 3 a la web sin mayores complicaciones.

Otro problema bastante claro es la falta de organización del equipo y la falta de tiempo. Dejar el trabajo para el término de semestre es una mala idea.

Finalmente el problema mas grande que aun no podemos resolver fue la utilización de jenkins con Selenium, ya que al tratar de correr y ejecutar las pruebas no podemos ver la interfaz web, por lo que las pruebas no se ejecutan.

5. Conclusión

Jenkins es un excelente gestor para temas de CI/CD e integración de otro tipos de frameworks de terceros, u otros no necesariamente relacionados a testing o CI/CD, como generación de reportes o avisos. Esto igual es un punto en contra, dado que hay varios plugins des actualizados o "deprecados" que la gente ya no mantiene, por lo que no sabes si funcionarán o no y su documentación no siempre está al día.

Pese a ello, Jenkins no es complejo de usar, y se complementa de manera increíble con GitHub, pero igual prefiero utilizar los Git Runners que funcionan para su tarea y nada más. Usar Jenkins en proyectos pequeños puede ser utilizar una herramienta demasiado poderosa e incluso innecesaria a veces.